Red neuronal de tres capas ocultas con TensorFlow + Keras

Diseño, arquitectura, entrenamiento y evaluación

CADI Deep Learning — Conceptos

Estudiante: Ruby Dayana Cárdenas Gómez

Docente: Nidia Stella García Roa

Institución: Universidad de Cundinamarca

Programa: CADI Deep Learning — Conceptos · Especialización en Inteligencia Artificial

Fecha: Octubre de 2026

Cuaderno de Google Colab: **https://colab.research.google.com/drive/ENLACE**

1. Introducción

En los cuadernos de Redes Neuronales Básicas y Redes Neuronales II construí redes pequeñas, primero a mano con NumPy y luego con Keras y PyTorch sobre MNIST. En este ejercicio doy un paso más: una red neuronal densa con tres capas ocultas aplicada a Fashion-MNIST, un conjunto de 70 000 imágenes de prendas de ropa que es más difícil que los dígitos porque varias clases se parecen entre sí.

El objetivo es triple: diseñar y explicar la arquitectura, representar gráficamente cómo se relacionan las capas, y entrenar y evaluar la red con métricas y gráficas. Todo el código está en el cuaderno de Google Colab `RedNeuronalTresCapas_RCardenas.ipynb`, cuyo enlace aparece en la portada y en la sección 7.

2. Marco conceptual

Una red neuronal artificial es una función compuesta por capas de unidades (neuronas) que reciben entradas, las combinan linealmente con unos pesos y un sesgo, y aplican una función de activación no lineal (Goodfellow et al., 2016). En una **capa densa** cada neurona está conectada con todas las de la capa anterior; apilar varias capas permite que la red aprenda representaciones cada vez más abstractas: las primeras capas detectan bordes y manchas, las siguientes combinan esos patrones en formas y la última los convierte en una decisión (LeCun et al., 2015).

La activación **ReLU** (max(0, z)) se usa en las capas ocultas porque es simple, evita que el gradiente se desvanezca y deja apagadas las neuronas que no aportan. La **Softmax** en la salida convierte 10 números en probabilidades que suman 1. El entrenamiento ajusta los pesos con retropropagación del error y el optimizador **Adam**, que adapta la tasa de aprendizaje de cada peso (Chollet, 2021).

3. Datos y preprocesamiento

Fashion-MNIST tiene imágenes de 28 × 28 píxeles en escala de grises y 10 clases (camiseta, pantalón, suéter, vestido, abrigo, sandalia, camisa, tenis, bolso y bota). Cada imagen se aplanó a un vector de 784 valores y los píxeles se escalaron de 0-255 a 0-1 para que el entrenamiento sea estable. Separé 10 000 imágenes del conjunto de entrenamiento como validación, de modo que quedaron 50 000 para entrenar, 10 000 para validar y 10 000 para la prueba final.

2. Los datos: Fashion-MNIST

70 000 imágenes en escala de grises de 28 × 28 píxeles, repartidas en 10 clases de prendas. Cada imagen se convierte en un vector de 784 números (28 × 28) y los píxeles se escalan de 0-255 a 0-1 para que la red entrene de forma estable.

[2]

```
(X_ent, Y_ent), (X_pru, Y_pru) = keras.datasets.fashion_mnist.load_data()
CLASES = ["Camiseta", "Pantalón", "Suéter", "Vestido", "Abrigo", "Sandalia", "Camisa", "Tenis", "Bolso", "Bota"]

# Aplanar y escalar
X_ent = X_ent.reshape(-1, 784).astype("float32") / 255.0
X_pru = X_pru.reshape(-1, 784).astype("float32") / 255.0

# Separo 10 000 imágenes de entrenamiento para validación
X_val, Y_val = X_ent[:10000], Y_ent[:10000]
X_ent, Y_ent = X_ent[10000:], Y_ent[10000:]

print("Entrenamiento:", X_ent.shape, "| Validación:", X_val.shape, "| Prueba:", X_pru.shape)
```

```
Entrenamiento: (50000, 784) | Validación: (10000, 784) | Prueba: (10000, 784)
```

Captura 1. Carga, aplanado y escalado de los datos; división en entrenamiento, validación y prueba.



Figura 1. Ejemplos del conjunto de entrenamiento con su clase.

4. Arquitectura de la red

La red tiene una capa de entrada de 784 neuronas, tres capas ocultas de 128, 64 y 32 neuronas con activación ReLU, y una capa de salida de 10 neuronas con Softmax. Elegí tamaños decrecientes para que la información se vaya comprimiendo: de 784 píxeles a 32 características antes de la decisión final.

Capa	Neuronas	Activación	Parámetros	Qué hace
Entrada	784	—	0	Recibe los píxeles de la imagen
Oculta 1	128	ReLU	100 480	Detecta patrones simples: bordes y manchas
Oculta 2	64	ReLU	8 256	Combina los patrones en formas
Oculta 3	32	ReLU	2 080	Resume la información en pocas características
Salida	10	Softmax	330	Probabilidad de cada prenda
Total			111 146	

Tabla 1. Arquitectura de la red. Los parámetros de cada capa son entradas × neuronas + neuronas (sesgos).

3. Arquitectura de la red

Cada neurona de una capa está conectada con todas las neuronas de la siguiente (por eso se llaman capas densas). Cada conexión tiene un peso que la red ajusta durante el entrenamiento.

[4]

```
modelo = keras.Sequential([
    keras.layers.Input(shape=(784,), name="entrada"),
    keras.layers.Dense(128, activation="relu", name="oculta_1"),
    keras.layers.Dense(64, activation="relu", name="oculta_2"),
    keras.layers.Dense(32, activation="relu", name="oculta_3"),
    keras.layers.Dense(10, activation="softmax", name="salida"),
], name="red_tres_capas")

modelo.compile(optimizer=keras.optimizers.Adam(learning_rate=0.001),
               loss="sparse_categorical_crossentropy",
               metrics=["accuracy"])
modelo.summary()
```

```
Model: "red_tres_capas"
```

Layer (type)	Output Shape	Param #
oculta_1 (Dense)	(None, 128)	100,480
oculta_2 (Dense)	(None, 64)	8,256
oculta_3 (Dense)	(None, 32)	2,080
salida (Dense)	(None, 10)	330

```
Total params: 111,146 (434.16 KB)
Trainable params: 111,146 (434.16 KB)
Non-trainable params: 0 (0.00 B)
```

Captura 2. Definición del modelo en Keras y resumen con el número de parámetros por capa.

4.1 Gráfica de la relación entre las capas

La Figura 2 muestra la red como un grafo: cada columna es una capa, cada círculo una neurona (se dibujan 12 por capa y los puntos suspensivos indican que hay más) y cada línea una conexión con su peso. Entre capas consecutivas aparece el número total de pesos: la mayor parte de los parámetros (100 352) está entre la entrada y la primera capa oculta, porque ahí se conectan 784 neuronas con 128. Las capas siguientes son mucho más baratas. La Figura 3 es la misma arquitectura dibujada por Keras, con la forma del tensor que entra y sale de cada capa.

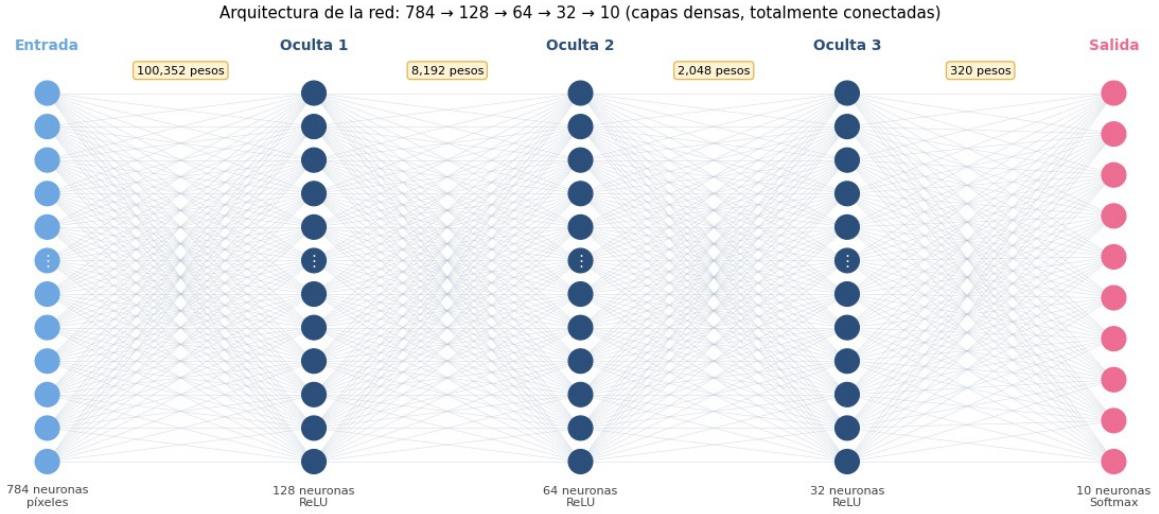


Figura 2. Relación entre las capas: red totalmente conectada 784 → 128 → 64 → 32 → 10, con el número de pesos entre cada par de capas.

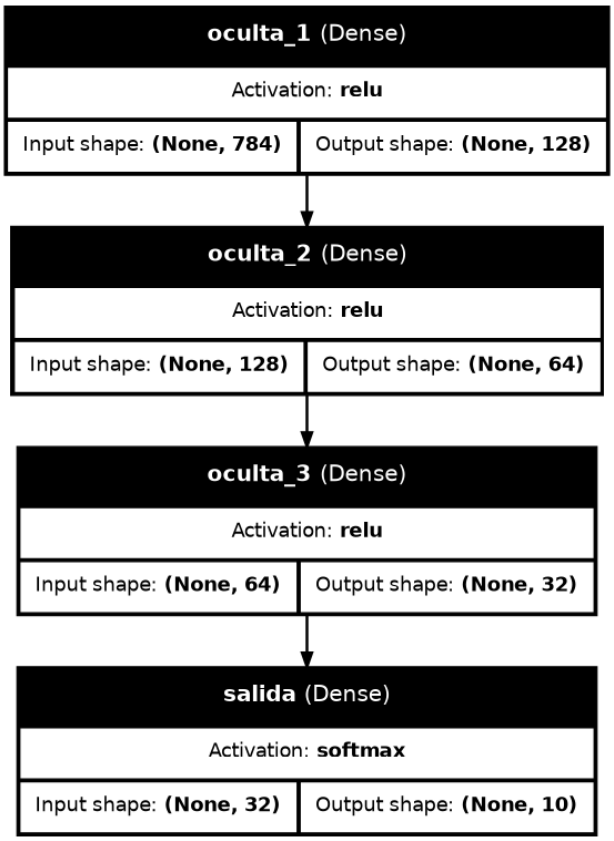


Figura 3. Diagrama de bloques generado con keras.utils.plot_model.

5. Entrenamiento

Entrené con lotes de 128 imágenes, optimizador Adam (tasa 0,001) y pérdida de entropía cruzada, con un máximo de 20 épocas y Early Stopping vigilando la pérdida de validación con paciencia de 4 épocas. La red se detuvo en la época 14 y conservó los pesos de la mejor época. La exactitud de entrenamiento llegó a 91,66 % y la de validación a 88,35 %.

4. Entrenamiento

Entreno 20 épocas con lotes de 128 imágenes, vigilando el conjunto de validación. Uso *Early Stopping* para detenerme si el error de validación deja de mejorar y quedarme con los mejores pesos.

[7]

```
parada = keras.callbacks.EarlyStopping(monitor="val_loss", patience=4, restore_best_weights=True)

historia = modelo.fit(X_ent, Y_ent,
                      validation_data=(X_val, Y_val),
                      epochs=20, batch_size=128,
                      callbacks=[parada], verbose=2)
```

```
Epoch 1/20
391/391 - 3s - 7ms/step - accuracy: 0.7954 - loss: 0.5985 - val_accuracy: 0.8275 - val_loss: 0.4782
Epoch 2/20
391/391 - 2s - 6ms/step - accuracy: 0.8555 - loss: 0.4051 - val_accuracy: 0.8460 - val_loss: 0.4253
Epoch 3/20
391/391 - 1s - 3ms/step - accuracy: 0.8687 - loss: 0.3596 - val_accuracy: 0.8491 - val_loss: 0.4125
Epoch 4/20
391/391 - 1s - 3ms/step - accuracy: 0.8780 - loss: 0.3338 - val_accuracy: 0.8549 - val_loss: 0.3961
Epoch 5/20
391/391 - 1s - 4ms/step - accuracy: 0.8845 - loss: 0.3143 - val_accuracy: 0.8646 - val_loss: 0.3677
Epoch 6/20
391/391 - 3s - 6ms/step - accuracy: 0.8900 - loss: 0.2980 - val_accuracy: 0.8668 - val_loss: 0.3541
Epoch 7/20
391/391 - 3s - 7ms/step - accuracy: 0.8936 - loss: 0.2866 - val_accuracy: 0.8734 - val_loss: 0.3401
Epoch 8/20
391/391 - 2s - 6ms/step - accuracy: 0.8980 - loss: 0.2745 - val_accuracy: 0.8769 - val_loss: 0.3309
Epoch 9/20
391/391 - 1s - 4ms/step - accuracy: 0.9025 - loss: 0.2630 - val_accuracy: 0.8798 - val_loss: 0.3320
Epoch 10/20
391/391 - 1s - 3ms/step - accuracy: 0.9061 - loss: 0.2535 - val_accuracy: 0.8808 - val_loss: 0.3282
Epoch 11/20
391/391 - 1s - 3ms/step - accuracy: 0.9090 - loss: 0.2446 - val_accuracy: 0.8778 - val_loss: 0.3340
Epoch 12/20
391/391 - 3s - 7ms/step - accuracy: 0.9128 - loss: 0.2348 - val_accuracy: 0.8787 - val_loss: 0.3388
Epoch 13/20
391/391 - 3s - 6ms/step - accuracy: 0.9149 - loss: 0.2288 - val_accuracy: 0.8807 - val_loss: 0.3315
Epoch 14/20
391/391 - 3s - 6ms/step - accuracy: 0.9166 - loss: 0.2236 - val_accuracy: 0.8835 - val_loss: 0.3335
```

Captura 3. Entrenamiento con Early Stopping (se muestran las primeras épocas).

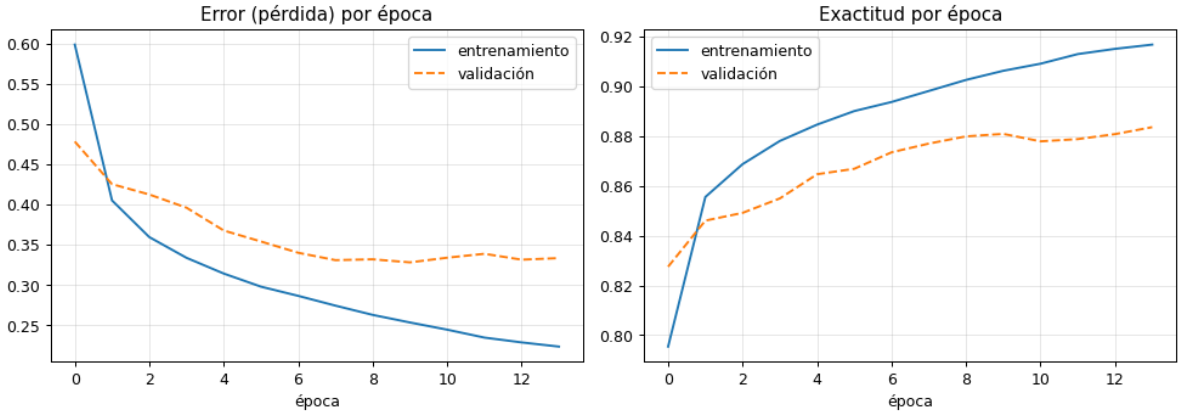


Figura 4. Curvas de pérdida y exactitud por época en entrenamiento y validación.

Las curvas cuentan la historia completa: la pérdida de entrenamiento baja de forma continua, pero la de validación se estabiliza alrededor de la época 10 (0,33) y deja de mejorar. A partir de ahí la red sigue memorizando el entrenamiento sin aprender nada útil para datos nuevos; es el inicio del sobreajuste, y el Early Stopping lo detectó y cortó el entrenamiento cuatro épocas después del mejor punto.

6. Resultados

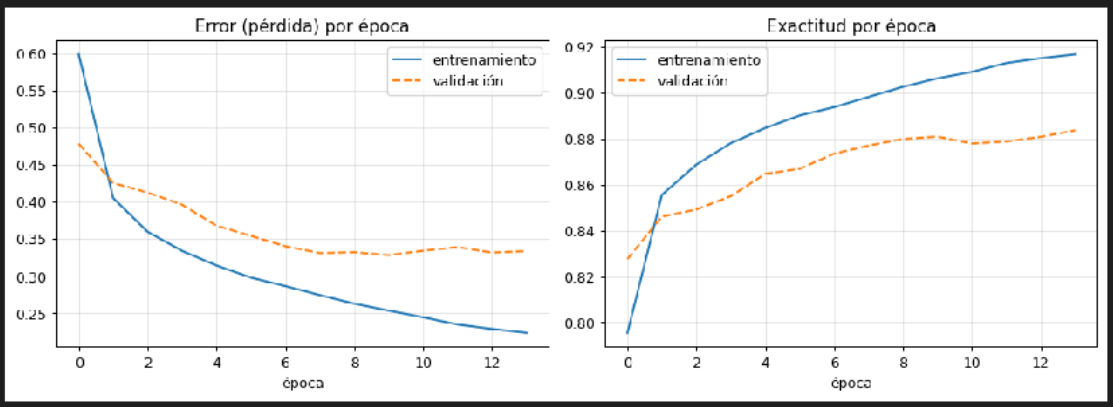
Sobre las 10 000 imágenes de prueba, que la red nunca vio, la exactitud fue de **87,26 %** con una pérdida de 0,357. El reporte por clase muestra que el desempeño no es uniforme: pantalón (F1 0,979), bolso (0,966) y

sandalia (0,958) se reconocen casi perfectamente, mientras que camisa (0,690), suéter (0,746) y abrigo (0,786) son las más difíciles.

[8]

```
h = historia.history
fig, ejes = plt.subplots(1, 2, figsize=(11, 4))
ejes[0].plot(h["loss"], label="entrenamiento"); ejes[0].plot(h["val_loss"], "--", label="validación")
ejes[0].set_title("Error (pérdida) por época"); ejes[0].set_xlabel("época"); ejes[0].legend();
ejes[0].grid(alpha=0.3)
ejes[1].plot(h["accuracy"], label="entrenamiento"); ejes[1].plot(h["val_accuracy"], "--", label="validación")
ejes[1].set_title("Exactitud por época"); ejes[1].set_xlabel("época"); ejes[1].legend(); ejes[1].grid(alpha=0.3)
plt.tight_layout(); plt.show()
print(f"Épocas entrenadas: {len(h['loss'])}")
print(f"Mejor exactitud en validación: {max(h['val_accuracy'])*100:.2f} %")
```

```
Épocas entrenadas: 14
Mejor exactitud en validación: 88.35 %
```



[9]

```
from sklearn.metrics import classification_report, confusion_matrix, ConfusionMatrixDisplay

perdida, exactitud = modelo.evaluate(X_pru, Y_pru, verbose=0)
print(f"Exactitud en prueba: {exactitud*100:.2f} %   |   Pérdida: {perdida:.4f}\n")

predicciones = np.argmax(modelo.predict(X_pru, verbose=0), axis=1)
print(classification_report(Y_pru, predicciones, target_names=CLASES, digits=3))
```

```
Exactitud en prueba: 87.26 %   |   Pérdida: 0.3566

              precision    recall  f1-score   support

    Camiseta      0.835     0.823     0.829      1000
    Pantalón      0.993     0.965     0.979      1000
      Suéter      0.878     0.649     0.746      1000
     Vestido      0.878     0.886     0.882      1000
      Abrigo      0.747     0.830     0.786      1000
    Sandalia      0.977     0.940     0.958      1000
      Camisa      0.642     0.744     0.690      1000
       Tenis      0.943     0.945     0.944      1000
       Bolso      0.960     0.972     0.966      1000
        Bota      0.927     0.972     0.949      1000

    accuracy                          0.873     10000
   macro avg      0.878     0.873     0.873     10000
weighted avg      0.878     0.873     0.873     10000
```

Captura 4. Evaluación en prueba y reporte de precisión, recall y F1 por clase.

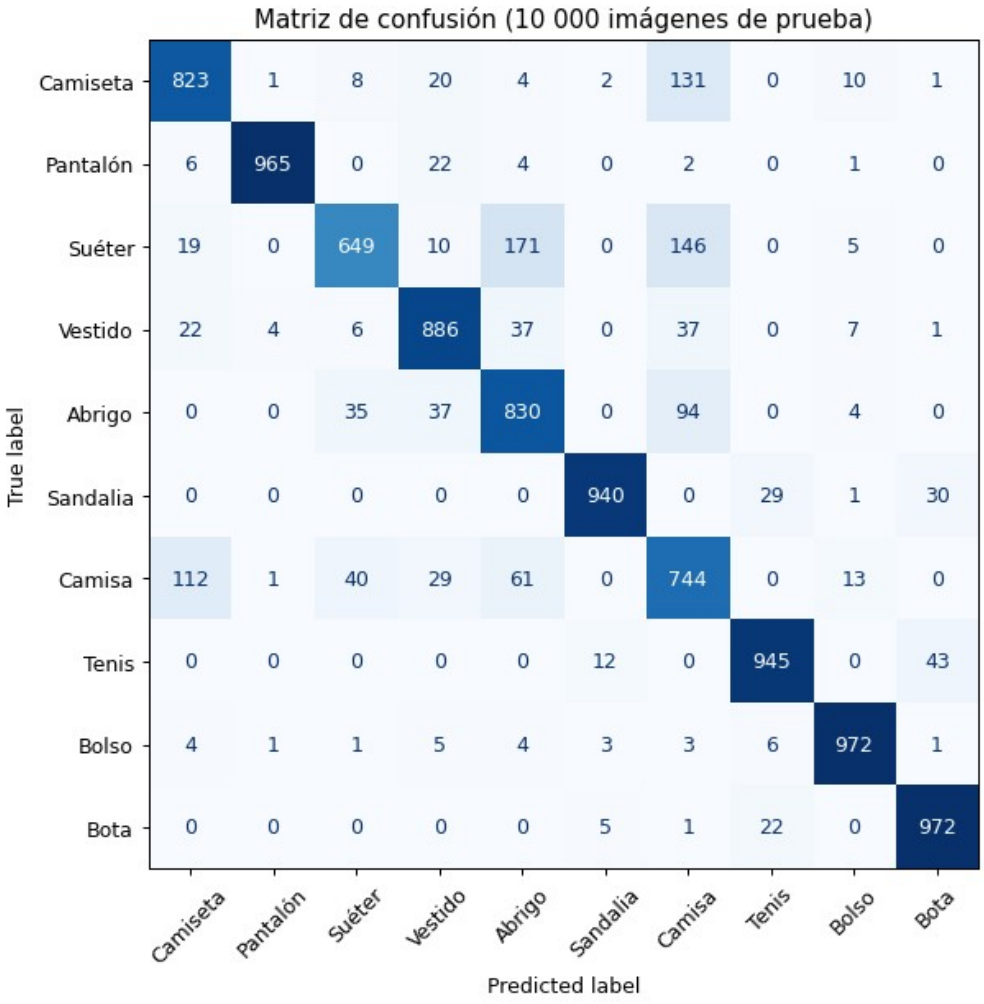


Figura 5. Matriz de confusión sobre el conjunto de prueba.

La matriz de confusión explica los números: las confusiones se concentran entre prendas que se parecen de lejos. El suéter se confunde con el abrigo y la camisa; la camisa con la camiseta, el suéter y el abrigo. Pantalón, bolso y bota apenas tienen errores porque su silueta no se parece a la de ninguna otra clase. Una red densa mira los píxeles uno a uno y no la estructura espacial, por eso le cuesta distinguir prendas que solo difieren en detalles como cuello, botones o largo de manga.

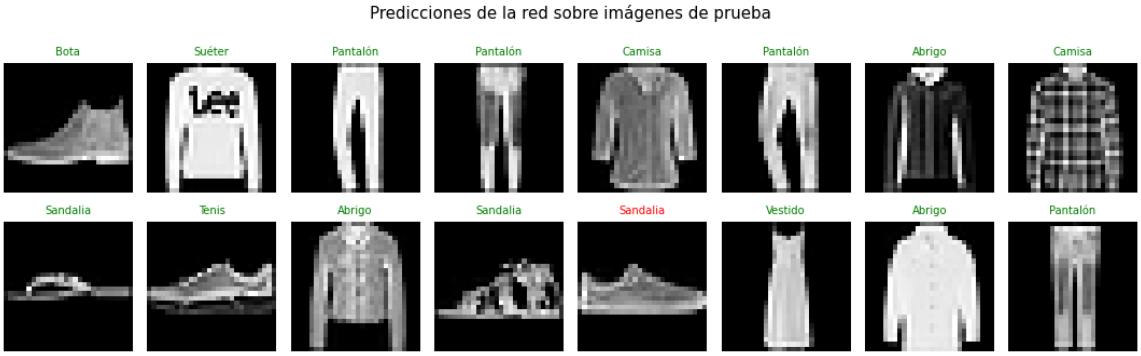


Figura 6. Predicciones sobre imágenes de prueba (verde: acierto, rojo: error).

6.1 Qué ocurre dentro de la red

Para ver la relación entre capas en funcionamiento, pasé una sola imagen (una bota) por la red y grafiqué la activación de cada capa. De las 128 neuronas de la primera capa oculta se encendieron 60; de las 64 de la segunda, 34; y de las 32 de la tercera, 15. La información se va concentrando en menos neuronas hasta que la salida asigna 99,4 % de probabilidad a la clase correcta. Esto confirma la idea de la Tabla 1: cada capa resume lo que recibe de la anterior.

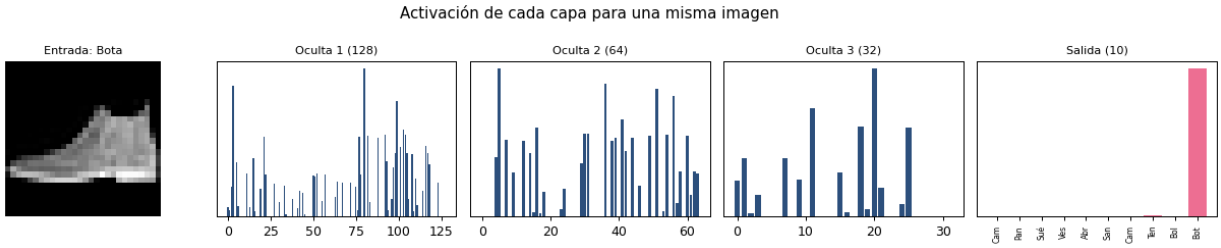
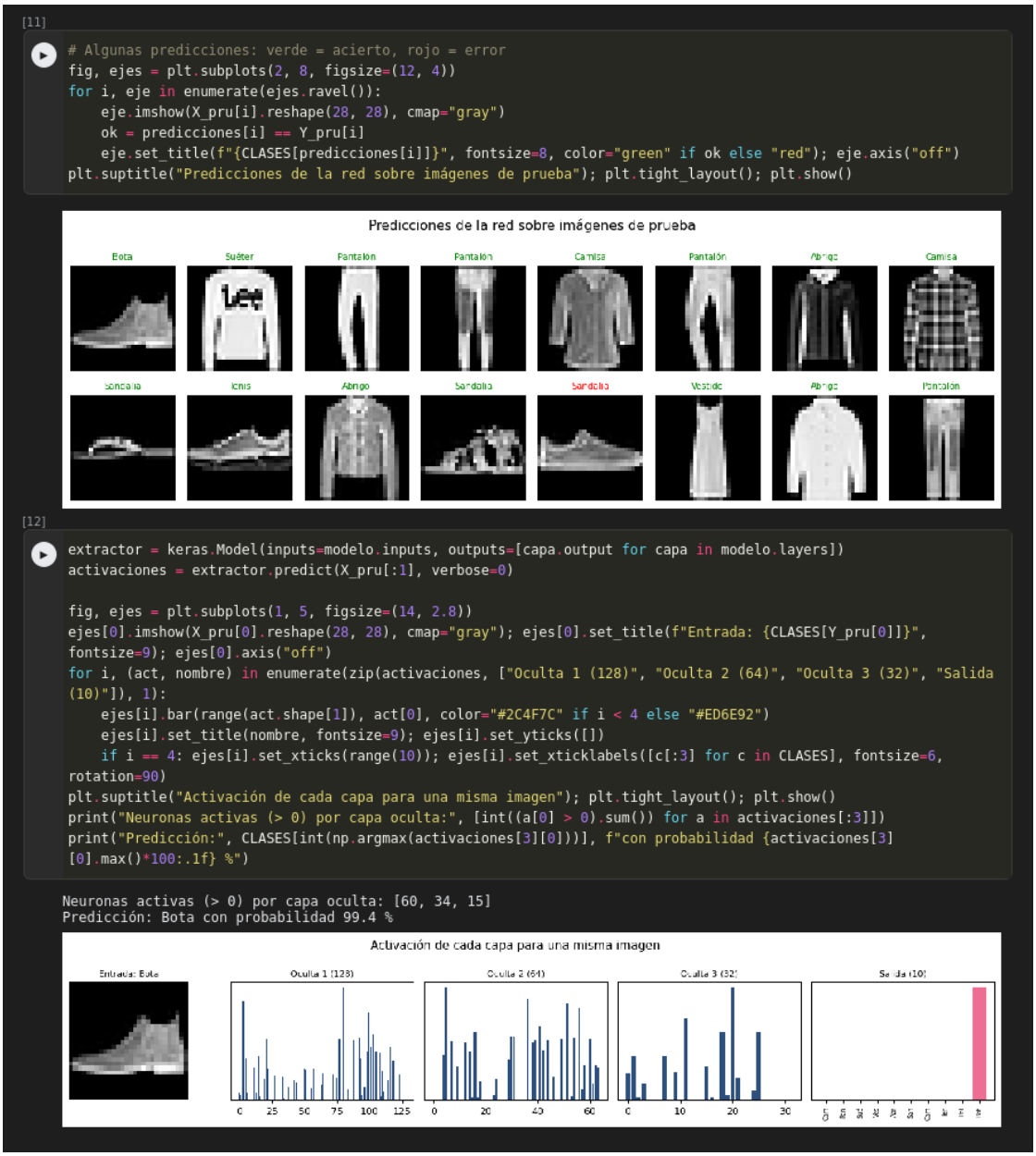


Figura 7. Activación de cada capa para una misma imagen de entrada.



Captura 5. Extracción de las activaciones intermedias con un modelo auxiliar de Keras.

7. Enlace al cuaderno de Google Colab

Cuaderno con todo el código, las gráficas y las salidas: **https://colab.research.google.com/drive/ENLACE**. Está construido con TensorFlow 2.21 y Keras 3; se ejecuta de principio a fin con “Ejecutar todo” en unos tres minutos y, gracias a la semilla fija, reproduce los mismos resultados.

8. Conclusiones

- Una red de tres capas ocultas con solo 111 146 parámetros clasifica correctamente el 87,26 % de 10 000 prendas que nunca vio. Casi todo el costo en parámetros está entre la entrada y la primera capa oculta (100 352 pesos); las capas profundas son baratas, pero son las que convierten patrones simples en decisiones.

- La relación entre capas es de compresión progresiva: 784 → 128 → 64 → 32 → 10. Al observar las activaciones se ve que cada capa enciende menos neuronas que la anterior, hasta que la salida concentra la probabilidad en una sola clase.
- Las curvas de entrenamiento son la herramienta de diagnóstico: cuando la pérdida de validación deja de bajar mientras la de entrenamiento sigue cayendo, la red está memorizando. Early Stopping convirtió esa observación en una regla automática.
- La exactitud global esconde diferencias grandes entre clases. La matriz de confusión muestra que el 13 % de error está casi todo en cuatro prendas parecidas (camisa, camiseta, suéter, abrigo); un modelo convolucional, que sí mira la estructura espacial, sería el siguiente paso natural.
- Comparado con mis cuadernos anteriores sobre MNIST, Fashion-MNIST deja claro que el mismo tipo de red rinde distinto según la dificultad de los datos, y que diseñar la arquitectura (cuántas capas, cuántas neuronas, qué activación) es una decisión que hay que justificar y evaluar, no un valor por defecto.

Referencias

Abadi, M., Agarwal, A., Barham, P., Brevdo, E., Chen, Z., Citro, C., … Zheng, X. (2016). *TensorFlow: Large-scale machine learning on heterogeneous distributed systems*. arXiv. https://doi.org/10.48550/arXiv.1603.04467

Chollet, F. (2021). *Deep learning with Python* (2.ª ed.). Manning.

Goodfellow, I., Bengio, Y., & Courville, A. (2016). *Deep learning*. MIT Press.

LeCun, Y., Bengio, Y., & Hinton, G. (2015). Deep learning. *Nature*, *521*(7553), 436–444. https://doi.org/10.1038/nature14539

Xiao, H., Rasul, K., & Vollgraf, R. (2017). *Fashion-MNIST: A novel image dataset for benchmarking machine learning algorithms*. arXiv. https://doi.org/10.48550/arXiv.1708.07747